Escuela de
Ciencia y Tecnología
ECyT_UNSAM

TPI - IA Monitoreo Cirugías Cardíacas

Verificación y Validación de software

Equipo: Deploy & Pray

Turno: Noche

Integrantes:

- *Samuel Trejo*
- *Camila Blesa*
- *Ian Gucevich*

Profesores:

- *Ing Di Ludovico Maximiliano*
- *Ing Millan Alejandro*

Índice de contenidos

1. Estrategia y enfoque....2
1.1 Objetivo....2
1.2 Alcance....2
1.3 Enfoque....3
1.4 Niveles y tipos de prueba....3
1.4.1 Metodologías aplicadas en el plan....4
1.4.2 Técnicas de diseño de casos de prueba....4
1.4.3 Técnicas de revisión y evaluación....5
1.4.4 Pruebas de recuperación ante desastres....5
1.4.5 Técnicas excluidas....5
1.5 Criterios de entrada y salida....5
1.6 Entornos de prueba y roles....6
1.7 Estimación y cronograma....7
1.8 Control de cambios....7
1.9 Riesgos....7
1.10 Gestión Visual de Tareas y Flujo de Trabajo....8
1.11 Indicadores Clave de Desempeño y Métricas de QA (KPIs)....8
2. Diseño y ejecución de pruebas....10
2.1 Rangos Clínicos y Reglas de Decisión....10
2.2 Técnica Pairwise (Microsoft PICT)....10
2.3 Umbrales de Riesgo y Disparo de Alertas (RF-06, RF-07 y RF-08)....11
2.4 Casos de Prueba de Caja Negra y Valores Límite....11
2.5 Cobertura JaCoCo....12
2.6 Testing de mutación PIT....13
2.7 Reporte y Gestión de Defectos....15
2.8 APIs con Postman....17
2.9 Automatización Web con Selenium WebDriver....18
3. Uso Asistido de Inteligencia Artificial....19
4. Informe Final....20
4.1 Balance de Cumplimiento de Criterios de Calidad (KPIs)....20
4.2 Riesgos Residuales y Limitaciones del Trabajo....20
4.3 Lecciones Aprendidas del Proceso de QA....21
4.4 Resultados finales de ejecución de pruebas....21
Cobertura código - JaCoCo....21
Cobertura controller - MockMvc....22
Mutation testing - PIT....22
Pruebas API - Postman....23
Pruebas end-to-end - Selenium....23
Pruebas BDD - Cucumber....24
Pruebas unitarias....24
5. Dictamen del Equipo de QA....24

1. Estrategia y enfoque

1.1 Objetivo

El objetivo propuesto es verificar el correcto funcionamiento del sistema de monitoreo cardíaco “CardioVigía” v2.4.0 del Instituto Cardiovascular del Plata (ICP), cumpliendo con los atributos de calidad y funcionalidades a alcanzar.

Como equipo de QA nos proponemos diseñar, justificar y ejecutar una estrategia de testing, seleccionando las técnicas, tipos de pruebas y criterio, según los riesgos del sistema y el alcance a definir. A partir de la evidencia, brindaremos un informe sobre el estado del sistema: si corresponde liberar la versión 2.4.0, liberarla con condiciones o no liberarla.

1.2 Alcance

Se evaluarán los siguientes puntos:

- Validación de señales (RF-01/02): Entrada de datos sobre signos vitales (frecuencia cardíaca, presión, saturación, temperatura) y rangos acotados
- Calidad de señal (RF-03): Detección y etiquetado de canales "no confiables" ante interrupciones de señal
- Roles, fases y registro de población (RF-04/05) : Cambios de fase quirúrgica con registro según rol (anestesiólogo/perfusionista) y filtros aplicados según perfil del paciente
- Transición de estados y partición según nivel de riesgo (RF-06)
- Alertas y escalamiento (RF-07/08): Supresión de fatiga de alarmas en el personal médico, regla del escalamiento automático por temporizador
- Modos ante fallos y recuperación (RF-09): Modo degradado si una fuente se pierde por tiempo determinado o si falla el componente de IA con recuperación de información previa a la falla (resguardo en base de datos o en memoria)
- Persistencia, Auditoría e Integración HCE (RF-10 y RF-11): Guardado de eventos en SQL o memoria, registro de auditoría e integración con la Historia Clínica Electrónica
- Atributos de calidad según el estándar ISO 25010:
    - Eficiencia: tiempo de respuesta del sistema ante operaciones
    - Disponibilidad: carga de horario donde el sistema deberá estar en funcionamiento 99,9% mensualmente sin intervenir en el monitoreo básico
    - Confiabilidad: tolerancia a fallos y recuperación ante desastres
    - Seguridad: protección de datos sensibles y autenticación en acciones
- Control de límites en umbrales: (Alto, Crítico) y bloqueo de creación o modificación del modelo sin autorización

Puntos excluidos:

- Uso de información sensible de pacientes reales. Excluido por regulaciones éticas y legales (Ley 25.326 de Protección de Datos Personales y Ley 26.529 de Derechos del Paciente); se utilizan únicamente datos ficticios
- Equipo médico en contexto hospitalario: No se incluirán equipos físicos ni consolas de bombas CEC reales
- Entrenamiento del algoritmo de Machine Learning
- Pruebas de estrés ante información masiva
- Maquetación web interactiva y desarrollo de interfaces gráficas

1.3 Enfoque

El enfoque de testing será combinado, utilizando técnicas de caja negra, caja gris y caja blanca para verificar el comportamiento externo del sistema y su funcionamiento interno; incluyendo pruebas automatizadas y pruebas manuales.

- Caja negra: se comprobarán las entradas y salidas del sistema sin analizar su implementación interna, verificando que los resultados coinciden con los requerimientos.
- Caja gris: se utilizará el conocimiento de la estructura del backend, las APIs y la persistencia para diseñar pruebas que permitan verificar la interacción entre componentes y la integridad de los datos almacenados.
- Caja blanca: se analizará la estructura interna del código para comprobar la cobertura de sentencias, decisiones y caminos mediante pruebas automatizadas.

Por otro lado, se realizará testing estático mediante revisión de requerimientos, Peer review y análisis del código.

1.4 Niveles y tipos de prueba

Nivel	**Verifica…**	**Herramientas**
Unitaria	Unidades de código aisladas con un comportamiento	JUnit 5
Integración	Interacción con otros componentes	Spring Boot, Mock
Sistema	El sistema completo contra los requerimientos especificados	PostMan
Aceptación	Criterios de aceptación establecidos por el equipo	Cucumber

Tipos de prueba	**Por qué se utilizan**
Funcional	Evalúa qué hace el sistema, comprobando que las funciones implementadas produzcan los resultados esperados como la validación de signos vitales, clasificación de riesgos y generación de alertas
No funcional	Evalúa atributos de calidad como eficiencia, disponibilidad, confiabilidad y seguridad, según ISO 25010.

Tipos de prueba	Por qué se utilizan
Humo	Comprueba después de cada compilación que las funcionalidades críticas del sistema funcionan correctamente, como recibir una señal válida, generar una estimación y emitir su alerta correspondiente.
Regresión	Verificar que las modificaciones o correcciones realizadas no afecten funcionalidades que anteriormente funcionaban correctamente.

1.4.1 Metodologías aplicadas en el plan

Red-Green-Refactor (TDD)

Se escribe una prueba que inicialmente falla (Red), se desarrolla el código mínimo indispensable para que la prueba pase (Green) y finalmente se refactoriza para mejorar el código (Refactor).

Given-When-Then (BDD)

Describe comportamientos mediante una estructura de contexto (Given), acción (When) y resultado esperado (Then), utilizando palabras clave para verificar los criterios de aceptación. Se utilizará Cucumber.

1.4.2 Técnicas de diseño de casos de prueba

Tipo	Requisito Asociado	A qué se aplica…	Herramientas
Partición de equivalencias	RF-01, 02, 03, 05, 06, 09	Valores válidos e inválidos de cada entrada (signos vitales, edad, métricas médicas, roles)	Planilla de partición
Valores límite	RF-01, 02, 03, 06, 07, 08, 09, 11	Pruebas sobre los límites de los rangos válido, cortes de riesgo y tiempos definidos	Planilla de valores límites
Tabla de decisiones	RF-06, 07, 08, 11	Combinaciones de condiciones que determinan las acciones del sistema, como la clasificación de riesgos, generación, supresión y escalamiento de alertas.	Planilla de tabla de decisiones con resultados
Transición de estados	RF-04, 09, 07	Cambios en el estado del paciente o cirugía (Fases quirúrgicas, modo normal/degradado, ciclo de alertas)	Planilla de transición
Pairwise	RF-05, 11	Cubre todas las combinaciones posibles de pares de valores entre parámetros	PICT Microsoft
Error Guessing	RF-03, 09, 10	Pruebas basadas en experiencia y los que se consideran errores más comunes en un sistema (Null, campos vacíos, tipeo incorrecto)	Lista de sospechas

1.4.3 Técnicas de revisión y evaluación

Técnica	A qué se aplica	Herramientas
Testing de mutación	Se introduce deliberadamente un defecto en el sistema (“mutante”) con modificaciones sintácticas pequeñas para estimar la capacidad de detección de defectos en un conjunto de pruebas	PIT
Cobertura de código	Permite medir qué partes del código fueron ejecutadas por las pruebas, considerando cobertura de sentencias, decisiones y caminos.	JUnit 5 / JaCoCo
Peer Review	Revisión entre colegas para detectar defectos en el código tomando notas y realizando una puesta en común	Notas

1.4.4 Pruebas de recuperación ante desastres

Se verifica la capacidad del sistema para recuperarse por pérdida de conexión o caída del módulo de IA con las siguientes herramientas: JUnit 5, Mockito, Spring Boot Test y Postman

1.4.5 Técnicas excluidas

Las siguientes técnicas se excluyen del alcance del plan por decisión del equipo, considerando los objetivos y recursos disponibles para el proyecto.

Técnica excluida	Motivo de exclusión
Sanity Test	El alcance del plan se encuentra cubierto por las pruebas de humo y regresión, por lo que no se consideró necesario incorporarlo como una prueba independiente
Prueba de configuración	No se contempla en el alcance probar sistemas operativos o navegadores para la ejecución del sistema ni configuraciones adicionales por usuario
Pruebas de estrés	El alcance se concentra en verificar el comportamiento esperado del backend. Se realizarán pruebas de rendimiento y carga para comprobar los tiempos de respuesta y el funcionamiento con la cantidad prevista de pacientes en simultáneo pero no valores extremos

1.5 Criterios de entrada y salida

Entrada

- Requerimientos funcionales (RF-01 a RF-11) y no funcionales (ISO 25010: Eficiencia, Disponibilidad, Confiabilidad, Seguridad, Usabilidad) identificados y descritos como casos de prueba
- Criterios de aceptación (UAT) definidos por RF/NFR con valores:
    - Tiempo respuesta estimación <2s (P95), <5s (máx)
    - Sensibilidad modelo ≥90% eventos críticos

    - ≤2 alertas falsas alta/crítica por hora por paciente
    - Disponibilidad 99.9% mensual sin perder monitoreo básico
    - Recuperación <30s tras reinicio
    - Umbrales: Alto 40-60%, Crítico ≥70%, doble aprobación
- Entornos de prueba configurados, con datos sintéticos generados
- Casos de prueba diseñados con técnicas especificadas (partición equivalencia, valores límite, tablas decisión, transición estados, pairwise, error guessing)
- Matriz de riesgos actualizada con mitigaciones y casos de prueba asociados

Salida

- Ejecución completa de la suite (unitarias, integración, sistema, aceptación, API, web, rendimiento, seguridad) con evidencia (logs, reportes, capturas)
- Cobertura JaCoCo: ≥ 80% línea, ≥ 75% decisión, ≥ 70% caminos en código crítico
- Mutation Score PIT: ≥ 90% (mutantes eliminados / totales) con sobrevivientes documentados
- Resultados: 100% casos críticos aprobados, ≥ 95% casos totales aprobados, 0 defectos críticos abiertos, ≤ 2 defectos altos abiertos
- Defectos reportados en Jira con: severidad (Crítico/Alto/Medio/Bajo), prioridad, evidencia, estado (Nuevo - En Progreso - Verificado - Cerrado)
- Matriz de trazabilidad: RF/NFR, casos de prueba, defectos, ejecución Automatizada ≥ 85% casos automatizados
- Pruebas de rendimiento: tiempo respuesta <2s (P95) / <5s (máx); carga 24 pacientes simultáneos (16 + 50% margen); recuperación <30s
- Pruebas de seguridad: autenticación/reautenticación acciones sin bloquear visualización de signos vitales
- Definition of Done cumplido para todos los ítems (código revisado, tests pasan, cobertura, mutación, trazabilidad)
- Informe final: KPIs vs alcance, riesgos residuales, limitaciones, recomendación fundamentada (Liberar / Liberar con condiciones / No liberar)

1.6 Entornos de prueba y roles

1. **Desarrollo**: Código local con datos ficticios para correr pruebas unitarias.
2. **Testing / QA**: Pruebas formales sobre funcionalidades del sistema.
3. **Staging**: Réplica fiel de producción para la validación final.
4. Producción: Ambiente real donde se utilizará el sistema ***(No está dentro del alcance de nuestro proyecto).***

Distribución de Roles del Equipo:

- **Camila Blesa**: QA Lead & Automation Engineer (Diseño de estrategia, pruebas de regresión y automatización Cucumber/JUnit 5)
- **Samuel Trejo**: Quality Assurance Analyst & Structural Tester (Técnicas de caja negra/blanca, cobertura JaCoCo y pruebas de mutación PIT)
- **Ian Gucevich**: Backend Test Engineer & Performance Tester (Pruebas de API Postman, evaluación de rendimiento y recuperación ante fallos)

1.7 Estimación y cronograma

Período	Alcance
9/10 hasta 30/10	Plan de pruebas, definición de técnicas, herramientas a utilizar e investigación sobre métricas, código mínimo
31/10 al 6/11	Ejecución de pruebas y registro de resultados
7/11 al 13/11	Informe final y preparación para exposición

1.8 Control de cambios

Las correcciones de código, técnica o caso de prueba se detallarán en el commit del repositorio en GitHub. Cambios en el documento o métricas definidas se informarán al equipo y quedará registrado tanto en el documento como en el código.

1.9 Riesgos

Falsos negativos en alertas críticas

Mitigación: Pruebas de valores límite, tablas de decisión y PIT

Fatiga de alarmas

Mitigación: Verificación de la calidad de señal, supresión de ruido y valores límites

Latencia en la emisión de alertas

Mitigación: Medición de tiempos de respuesta en cada endpoint con Postman

Múltiples alertas simultáneas

Mitigación: Priorización de alertas Crítico > Alto, pruebas de carga con 24 pacientes

Datos reales de pacientes

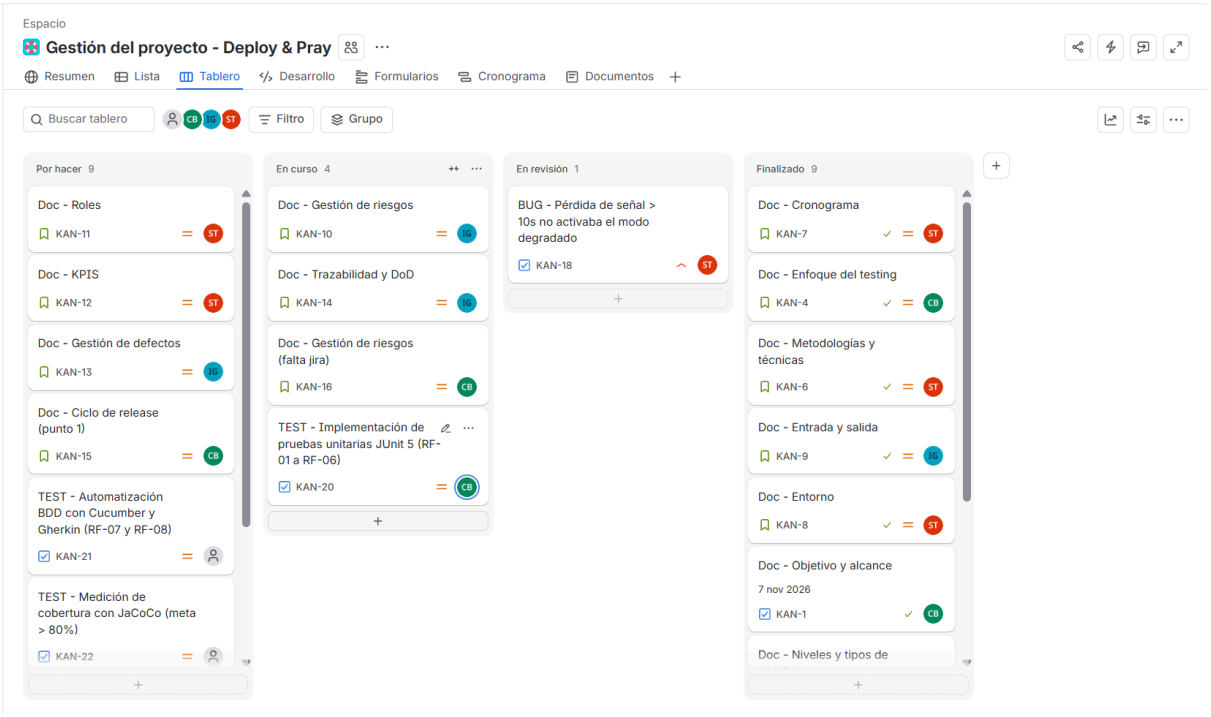
Mitigación: generador de datos sintéticos y pseudonimización

1.10 Gestión Visual de Tareas y Flujo de Trabajo

Para la organización, seguimiento y trazabilidad del proyecto se implementó un tablero Kanban en Jira Software denominado "Gestión del proyecto - Deploy & Pray". Este tablero facilita el control visual del progreso de cada una de las entregables y tareas asociadas al Plan de Pruebas.

El flujo de trabajo se estructuró en las siguientes cuatro columnas:

- **Por hacer (To Do)**: Contiene tareas pendientes asignadas a los integrantes del equipo (ej. documentación de roles, definición de KPIs, ciclo de release)
- **En curso (In Progress)**: Tareas en proceso activo de desarrollo o ejecución (ej. gestión de riesgos, matriz de trazabilidad y definición de DoD)
- **En revisión (In Review)**: Tareas finalizadas pendientes de revisión cruzada o aprobación por parte del equipo
- **Finalizado (Done)**: Actividades completadas satisfactoriamente (ej. cronograma, enfoque de testing, metodologías, criterios de entrada/salida, entorno y objetivo/alcance)



1.11 Indicadores Clave de Desempeño y Métricas de QA (KPIs)

Con el propósito de evaluar de manera objetiva el progreso, la efectividad y la calidad integral del proceso de pruebas sobre el sistema CardioVigía v2.4.0, se han establecido Indicadores Clave de Desempeño (KPIs) y métricas cuantitativas. Estos indicadores definen los umbrales mínimos exigidos para el cumplimiento de la Definición de Hecho (Definition of Done - DoD) y sustentan la toma de decisiones basada en evidencia técnica previo a la liberación del software.

KPI / Métrica	Propósito / Qué mide	Fórmula de Cálculo	Meta Exigida (Criterio de Aceptación)
Cobertura de Código (Líneas)	Mide el porcentaje de líneas de código fuente ejecutables pasadas durante las pruebas unitarias e integración.	(Líneas ejecutadas / Líneas totales ejecutables) × 100	≥ 80% en código general
Cobertura de Ramas (Decisiones)	Evalúa la proporción de bifurcaciones condicionales (if, switch, etc.) ejercitadas en sus flujos verdadero y falso.	(Decisiones evaluadas / Decisiones totales) × 100	≥ 75% en lógica de negocio
Mutation Score (PITest)	Mide la efectividad y calidad de la suite de pruebas mediante la eliminación de fallas sintéticas (mutantes) introducidas.	(Mutantes eliminados / Mutantes totales) × 100	≥ 90% en código crítico
Tasa de Aprobación de Pruebas	Porcentaje de casos de prueba ejecutados satisfactoriamente sobre el total planificado.	(Casos exitosos / Casos ejecutados) × 100	100% casos críticos / ≥ 95% casos totales
Densidad de Defectos Críticos Abiertos	Cantidad de fallos pendientes de resolución categorizados con severidad Crítica o Alta al finalizar el ciclo.	Conteo directo de bugs en Jira sin resolver	0 defectos críticos / ≤ 2 defectos altos
Nivel de Automatización de Requisitos	Porcentaje de requisitos funcionales (RF-01 a RF-11) validados mediante escenarios BDD/JUnit automatizados.	(Requisitos cubiertos automáticamente / RF totales) × 100	≥ 85% de cobertura automatizada
Tiempo de Respuesta (Latencia de API)	Tiempo transcurrido desde el envío de la solicitud HTTP hasta la recepción de la respuesta completa del backend.	Percentil 95 (P95) y Tiempo Máximo registrado en Postman	< 2s (P95) / < 5s (Máximo)

2. Diseño y ejecución de pruebas

En esta sección se informan las prácticas de cada una de las técnicas de prueba especificadas anteriormente para el backend del sistema.

2.1 Rangos Clínicos y Reglas de Decisión

Parámetro / Fuente	**Rango Normal Fisiológico**	**Rango Aceptado por el Sistema (RF-01/02)**	**Valores Límite / Fuera de Escala (Rechazados)**	**Acción del Sistema ante Dato Inválido**
Frecuencia Cardíaca (FC)	60 – 100 lpm	0 a 300 lpm	< 0 o > 300 lpm (ej: 301, 320)	Se descarta el valor por artefacto/ruido. No se usa para estimar
Presión Arterial Media (PAM)	70 – 105 mmHg	20 a 200 mmHg	< 20 o > 200 mmHg (ej: 19 o 201 mmHg)	Se descarta por señal aberrante
Saturación de Oxígeno (SpO2)	95 – 100 %	0 a 100 %	< 0 o > 100 % (ej: 101%)	Se rechaza por error de sensor
Temperatura Corporal	36.0 – 37.5 °C	15.0 a 43.0 °C *(admite hipotermia en CEC)*	< 15.0 o > 43.0 °C (ej: 14.9°C o 43.1°C)	Se descarta por fuera de rango físico
Calidad de Señal (RF-03)	Señal continua	Sin pérdida de señal o ≤ 15 segundos	> 15 segundos sin señal	El canal se marca como "No confiable" y suspende estimación
Población / Edad (RF-05)	N/A	18 a 90 años con perfil completo	< 18 años (menores) o > 90 años	No entra a IA; se mantiene en monitoreo básico

2.2 Técnica Pairwise (Microsoft PICT)

Para evaluar la combinación de parámetros clínicos y operativos (edad, perfil clínico, fase quirúrgica y rol médico), una prueba exhaustiva requeriría 3×2×3×3=54, combinaciones. Aplicando la técnica de caja negra combinatoria con la herramienta **Microsoft PICT**, se redujo la suite a **10 casos de prueba óptimos**, garantizando una cobertura del 100% de los pares de interacción y ahorrando un 81.5% de esfuerzo de prueba.

```
> .\pict.exe modelo_cardiovigia.txt
Edad    PerfilClinico   FaseQuirurgica  RolUsuario
Adulto_18_90    Completo        Postoperatorio  Anestesiologo
Mayor_90        Incompleto      Preoperatorio   Cirujano
Mayor_90        Completo        Intraoperatorio Anestesiologo
Menor_18        Incompleto      Postoperatorio  Perfusionista
Menor_18        Completo        Intraoperatorio Cirujano
Adulto_18_90    Incompleto      Postoperatorio  Cirujano
Adulto_18_90    Completo        Preoperatorio   Perfusionista
Adulto_18_90    Incompleto      Intraoperatorio Perfusionista
Mayor_90        Incompleto      Postoperatorio  Perfusionista
Menor_18        Incompleto      Preoperatorio   Anestesiologo
```

2.3 Umbrales de Riesgo y Disparo de Alertas (RF-06, RF-07 y RF-08)

Probabilidad estimada de complicación	**Nivel de riesgo**	**Tipo de Notificación**	**Destinatario Inicial**	**Regla de Escalamiento Automático (RF-08)**
P < 20%	Bajo	Ninguna (Monitoreo normal)	Panel general	No escala
20% ≤ P < 50%	Moderado	Indicación visual en panel (+ 3 factores determinantes)	Panel local quirófano	No escala
50% ≤ P < 80%	Alto	Alerta específica (+ 3 factores)	Anestesiólogo	Si no se reconoce en 60 segundos ➔ Escala al Cirujano
P ≥ 80%	Crítico	Alerta sonora + visual simultánea (+ 3 factores)	Anestesiólogo y Cirujano	Si no se reconoce en 30 segundos ➔ Escala al Coordinador UCI

Regla de Supresión de Alertas (RF-07): No se repite la misma alerta durante **2 minutos (120 s)** para evitar la *fatiga de alarmas*, salvo que el paciente empeore de nivel (ej. pase de Alto a Crítico)

Modo Degradado (RF-09): Si una fuente se pierde por **más de 10 segundos** o falla el módulo de IA, la pantalla muestra los últimos valores conocidos con su hora y avisa que no hay predicciones activas

2.4 Casos de Prueba de Caja Negra y Valores Límite

A continuación se detallan los casos de prueba diseñados aplicando partición de equivalencias y análisis de valores límite sobre los requisitos del sistema:

ID Caso	**Requisito**	**Entrada / Condición**	**Resultado Esperado**	**Técnica Aplicada**
CP-01	RF-01	FC = 75 lpm (Rango válido 0-300)	Dato aceptado y procesado por el modelo	Partición de Equivalencia
CP-02	RF-01	FC = 301 lpm	Dato descartado por fuera de escala; alerta de ruido	Valores Límite
CP-03	RF-03	Interrupción de señal por 16 segundos	Canal marcado como "No confiable"; suspensión de estimación	Valores Límite / Transición de Estados
CP-04	RF-06	Probabilidad de complicación P = 80%	Clasificación "Crítico"; alerta sonora + visual a Anestesiólogo y Cirujano	Tabla de Decisiones / Valores Límite
CP-05	RF-08	Alerta Crítica no reconocida en 31 segundos	Escalamiento automático al Coordinador UCI	Valores Límite / Temporizador

2.5 Cobertura JaCoCo

Results:

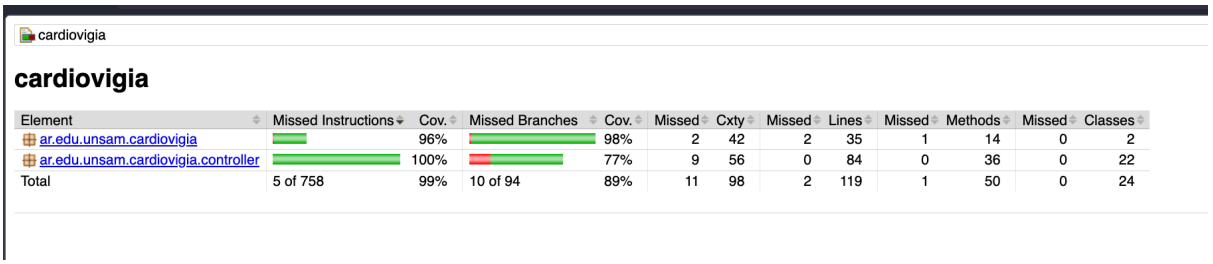
[INFO] Tests run: 136, Failures: 0, Errors: 0, Skipped: 0

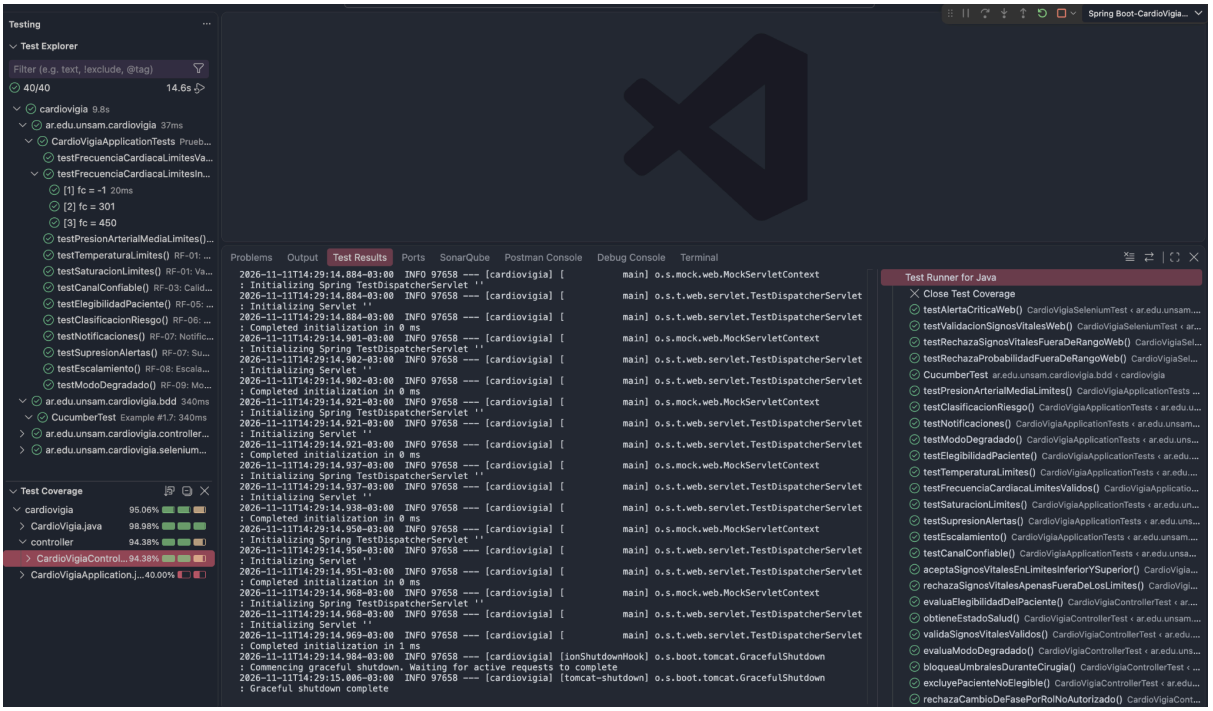
[INFO] Analyzed bundle 'cardiovigia' with 24 classes

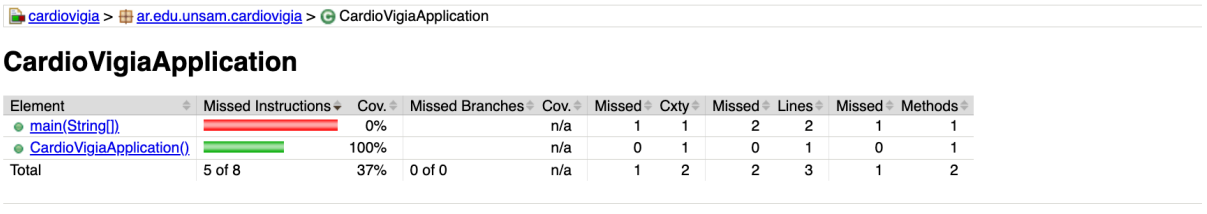
[INFO] BUILD SUCCESS

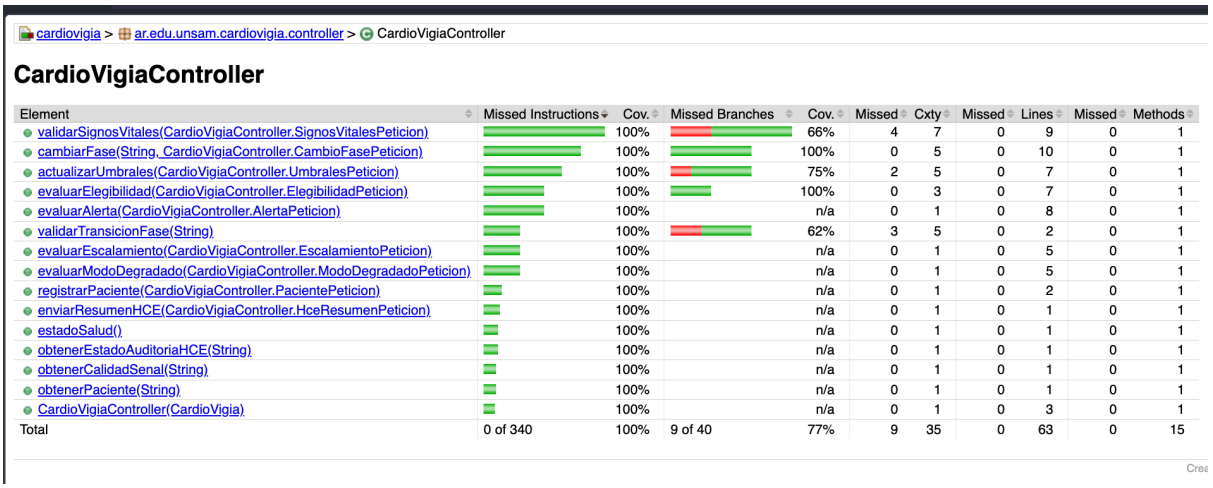
[INFO] Total time: 12.317 s

[INFO] Finished at: 2026-11-11T14:28:09-03:00









cardiovigia > ar.edu.unsam.cardiovigia.controller > CardioVigiaController

CardioVigiaController

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
validarSignosVitales(CardioVigiaController.SignosVitalesPeticion)		100%		66%	4	7	0	9	0	1
cambiarFase(String, CardioVigiaController.CambioFasePeticion)		100%		100%	0	5	0	10	0	1
actualizarUmbrales(CardioVigiaController.UmbralesPeticion)		100%		75%	2	5	0	7	0	1
evaluarElegibilidad(CardioVigiaController.ElegibilidadPeticion)		100%		100%	0	3	0	7	0	1
evaluarAlerta(CardioVigiaController.AlertaPeticion)		100%		n/a	0	1	0	8	0	1
validarTransicionFase(String)		100%		62%	3	5	0	2	0	1
evaluarEscalamiento(CardioVigiaController.EscalamientoPeticion)		100%		n/a	0	1	0	5	0	1
evaluarModoDegradado(CardioVigiaController.ModoDegradadoPeticion)		100%		n/a	0	1	0	5	0	1
registrarPaciente(CardioVigiaController.PacientePeticion)		100%		n/a	0	1	0	2	0	1
enviarResumenHCE(CardioVigiaController.HceResumenPeticion)		100%		n/a	0	1	0	1	0	1
estadoSalud()		100%		n/a	0	1	0	1	0	1
obtenerEstadoAuditoriaHCE(String)		100%		n/a	0	1	0	1	0	1
obtenerCalidadSenal(String)		100%		n/a	0	1	0	1	0	1
obtenerPaciente(String)		100%		n/a	0	1	0	1	0	1
CardioVigiaController(CardioVigia)		100%		n/a	0	1	0	3	0	1
Total	0 of 340	100%	9 of 40	77%	9	35	0	63	0	15

2.6 Testing de mutación PIT

El 96% del puntaje de mutación demuestra que la suite (14 unitarias, 21 MockMvc, 97 Cucumber y 4 Selenium) detecta eficazmente defectos introducidos en la lógica de decisión (umbrales de riesgo, alertas, escalamiento, supresión, modo degradado, elegibilidad, validación de signos vitales).

Cumple criterios de salida del plan de pruebas (1.5): Mutation Score ≥ 90%

Mutante 1

Detalle	Información
Método	`clasificarNivelRiesgo`
Línea	`CardioVigia.java:36`
Código	`if (probabilidad < 0.0 || probabilidad > 100)`
Mutante	ConditionalsBoundaryMutator (se cambió el límite condicional) `if (probabilidad <= 0.0 || probabilidad >= 100)`
Acción original	Rechazar probabilidades menores que `0.0` o mayores que `100.0`
Tests que cubren la línea	`clasificaProbabilidadesEnLimitesDeRiesgo`, `rechazaProbabilidadEnLimitesInvalidos`
Por qué sobrevive	No se prueban explícitamente ambos extremos válidos (`0.0` y `100.0`). PIT modifica el límite condicional y las pruebas actuales no distinguen ese cambio

Detalle	Información
Método	`actualizarUmbrales`

Detalle	Información
Línea	`CardioVigiaController.java:176`
Código	if (peticion.umbralAlto() < 40.0 || peticion.umbralAlto() > 60.0 || peticion.umbralCritico() < 70.0)
Mutante	ConditionalsBoundaryMutator (se cambió el límite condicional) if (peticion.umbralAlto() <= 40.0 || peticion.umbralAlto() >= 60.0 || peticion.umbralCritico() <= 70.0)
Acción original	Rechazar valores de `umbralAlto` menores que `40.0`, mayores que `60.0` o valores de `umbralCritico` menores que `70.0`
Tests que cubren la línea	`aceptaUmbralesValidos`, `rechazaUmbralesApenasFueraDeLosLimites`, `bloqueaUmbralesDuranteCirugia`
Por qué sobrevive	Las pruebas usan valores dentro del rango (`50.0`, `80.0`) y fuera del rango (`39.99`, `69.99`), pero no validan exactamente los límites `40.0`, `60.0` y `70.0`

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
2	100%	95/95	96%	104/108	96%	104/108

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
ar.edu.unsam.cardiovigia	1	100% 32/32	98% 65/66	98% 65/66
ar.edu.unsam.cardiovigia.controller	1	100% 63/63	93% 39/42	93% 39/42

- Project uses Spring, but the Arcmutate Spring plugin is not present.

Report generated by PIT 1.25.7 support

Enhanced functionality available at arcmutate.com

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
1	100%	32/32	98%	65/66	98%	65/66

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
ar.edu.unsam.cardiovigia	1	100% 32/32	98% 65/66	98% 65/66

- Project uses Spring, but the Arcmutate Spring plugin is not present.

Report generated by PIT 1.25.7 support

Enhanced functionality available at arcmutate.com

2.7 Reporte y Gestión de Defectos

A continuación se documentan los defectos identificados durante el ciclo de pruebas del sistema CardioVigía v2.4.0, confeccionados utilizando la plantilla oficial de Informe de Defectos de la cátedra de Verificación y Validación de Software (UNSAM). Todos los defectos fueron reportados en Jira, analizados, corregidos y validados mediante tests unitarios y de integración antes del cierre de esta versión.

Defecto 1: KAN-17 - Frecuencia Cardíaca de 320 lpm no es rechazada por fuera de escala

Campo	Detalle del Registro
Problema / Título	[KAN-17] Frecuencia cardíaca de 320 lpm es procesada por el modelo en lugar de ser descartada por fuera de escala (RF-01)
Fecha de apertura	02/11/2026
Fecha de cierre	05/11/2026
Acción realizada	1. Iniciar sesión en el sistema con rol Anestesiólogo. 2. Enviar mediante la API POST /api/signos-vitales una lectura de entrada con FC = 320 lpm (valor por fuera del rango aceptado 0-300 lpm). 3. Inspeccionar el objeto de respuesta del servicio de estimación de riesgo.
Resultado esperado	El sistema debe rechazar y descartar el valor de FC = 320 lpm por artefacto/ruido, no incluirlo en el algoritmo de IA y emitir una advertencia de señal fuera de escala según RF-01.
Resultado real	El valor 320 lpm es aceptado y procesado por el modelo de IA, calculando un riesgo distorsionado debido a una validación de límite superior incorrecta en la capa de servicio.
Mensaje de error	No se presenta mensaje de error (fallo silencioso en validación).
Gravedad	Alta. Procesar datos fisiológicos fuera de escala perturba el cálculo de probabilidad de complicación hemodinámica.
Prioridad	Alta. Afecta directamente el requisito central de validación de señales (RF-01).
Asignado a	Samuel Trejo (Structural Tester)
Resumen	Valores de FC superiores a 300 lpm no eran filtrados por una condición de frontera incorrecta en el método de sanitización de señales.
Descripción	Durante las pruebas con valores límite en el endpoint POST /api/signos-vitales, se constató que la lectura de FC = 320 lpm no fue descartada. El análisis de código reveló que la condición de validación utilizaba un operador disyuntivo incorrecto en lugar de evaluar estrictamente el límite superior de 300 lpm.
Estado	Corregido y Cerrado. Se corrigió la condición de frontera en el backend y se incorporó el test unitario testFrecuenciaCardiacaLimitesInvalidos() en JUnit 5 para asegurar su no regresión.

Defecto 2: KAN-18 - Pérdida de señal > 10s no activaba el modo degradado

Campo	Detalle del Registro
Problema / Título	[KAN-18] La pérdida de señal de una fuente por más de 10 segundos no activa automáticamente el modo degradado (RF-09)
Fecha de apertura	04/11/2026
Fecha de cierre	07/11/2026
Acción realizada	1. Simular la transmisión continua de datos del sensor de SpO2. 2. Interrumpir la fuente de datos por 12 segundos (> 10 s). 3. Consultar el estado del sistema mediante GET /api/modo-degradado/{pacienteId}.
Resultado esperado	El sistema debe conmutar al estado Modo Degradado, mantener visibles los últimos valores conocidos con su marca temporal y notificar que las predicciones de IA se encuentran pausadas (RF-09).
Resultado real	El sistema permaneció en modo normal de monitoreo sin conmutar al estado degradado, intentando ejecutar inferencias con datos incompletos.
Mensaje de error	No se presenta mensaje de error.
Gravedad	Crítica. Pone en riesgo la seguridad de la operación al no informar la pérdida de precisión por caída parcial de sensores.
Prioridad	Alta. Requiere resolución inmediata por comprometer la tolerancia a fallos del sistema.
Asignado a	Ian Gucevich (Backend Test Engineer)
Resumen	El temporizador de inactividad de señal no disparaba el cambio de estado debido a una descalibración en la tarea programada de verificación de canal.
Descripción	Al simular la pérdida prolongada de un canal por más de 10 segundos, la lógica de verificación mantenía el canal activo al no actualizar correctamente el timestamp del último paquete recibido en la caché local.
Estado	Corregido y Cerrado. Se actualizó el componente de gestión de estados de señal y se agregó el test de automatización testModoDegradado() para garantizar el cumplimiento de RF-09.

Defecto 3: KAN-19 - Alerta crítica no escala a los 30s exactos por condición de frontera

Campo	Detalle del Registro
Problema / Título	[KAN-19] Alerta crítica no escala al Coordinador UCI a los 30 segundos exactos por evaluación estricta (> 30s) (RF-08)
Fecha de apertura	06/11/2026
Fecha de cierre	08/11/2026

Campo	Detalle del Registro
Acción realizada	1. Generar una alerta de nivel Crítico (P ≥ 80%). 2. No reconocer la alerta durante 30 segundos exactos. 3. Verificar el destinatario de la notificación de escalamiento.
Resultado esperado	Al transcurrir 30 segundos sin acuse de recibo, la alerta debe escalar automáticamente hacia el Coordinador UCI (RF-08).
Resultado real	La alerta escalaba recién al segundo 31 debido a un operador de comparación mayor estricto (> 30) en la regla del temporizador.
Mensaje de error	No se presenta mensaje de error.
Gravedad	Alta. Retrasar 1 segundo el escalamiento de un evento crítico no cumple con la especificación de valores límite dada.
Prioridad	Media. El escalamiento finalmente ocurría, pero fuera del valor exacto especificado.
Asignado a	Camila Blesa (QA Lead & Automation Engineer)
Resumen	Error de condición de frontera (> 30 en lugar de >= 30) causaba un desfasaje de 1 segundo en el temporizador de escalamiento.
Descripción	En el análisis del test de valores límite para temporizadores se observó que la alerta no gatillaba el evento de escalamiento al cumplirse los 30.0s exactos, sino al alcanzar los 31.0s, producto de la comparación inclusiva no aplicada correctamente.
Estado	Corregido y Cerrado. Se actualizó la condición de frontera en la regla de negocio y se validó mediante el test unitario testEscalamiento() en JUnit 5.

2.8 APIs con Postman

Requisito	Variables	Ruta	Requests
RF-01	Frecuencia cardíaca	`/signos-vitales/validar`	2
RF-02	Presión arterial, SpO_2 y temperatura	`/signos-vitales/validar`	2
RF-03	Calidad de señal	`/calidad-senal/{id}`	1

RF-04	Fases quirúrgicas y roles	`/pacientes`
		`/pacientes/{id}`
		`/pacientes/{id}/fase`
			6

RF-05	Elegibilidad del paciente	`/pacientes/elegibilidad`	2
RF-06	Clasificación del riesgo	`/alertas/evaluar`	4
RF-07	Notificaciones	`/alertas/evaluar`	4

Requisito	Variables	Ruta	Requests
RF-08	Escalamiento	`/alertas/escalamiento`	2
RF-09	Modo degradado	`/modo-degradado`	2
RF-10	HCE	`/hce/resumen` `/hce/estado/{id}`	2
RF-11	Umbrales y bloqueo durante cirugía	`/config/umbrales`	2

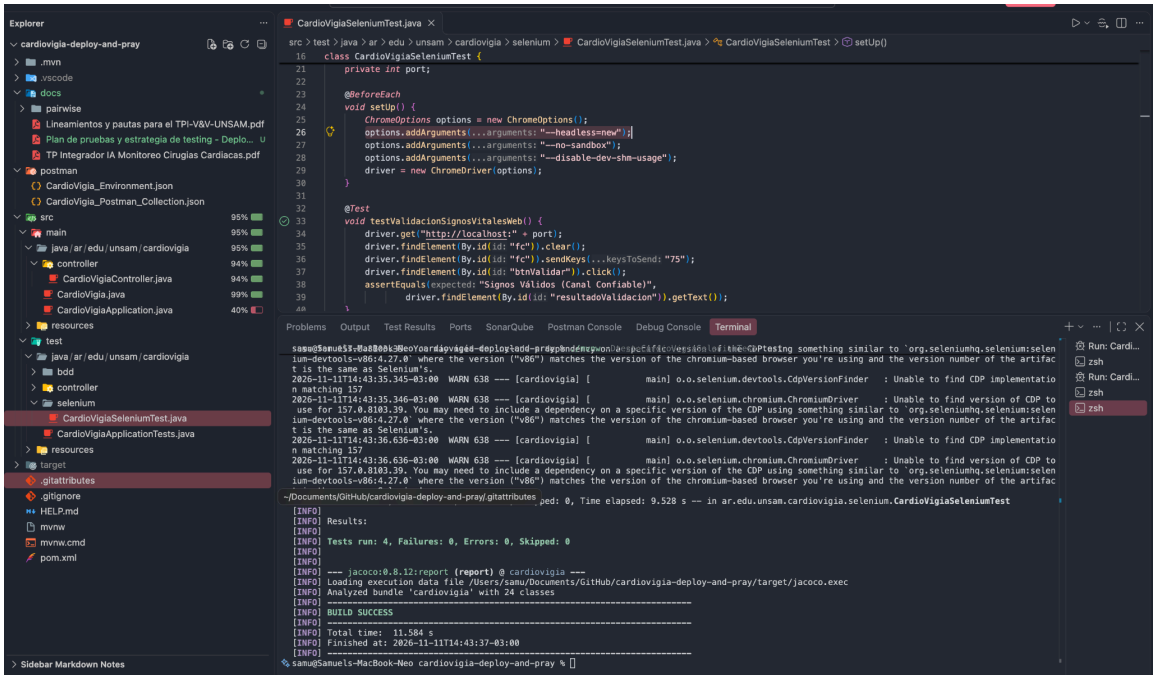
Algunas requests cubren más de un requisito funcional. Por este motivo, la suma de requests por requisito representa asignaciones de cobertura y no la cantidad de requests únicas de la colección. La colección contiene 26 requests únicas y 29 totales.

2.9 Automatización Web con Selenium WebDriver

Se implementaron pruebas automatizadas end-to-end (E2E) sobre la interfaz gráfica del monitor quirúrgico web (index.html) utilizando Selenium WebDriver 4.27.0 y ChromeDriver en entorno Spring Boot (@SpringBootTest). Se detallan los dos casos automatizados:

Verificación de ingreso de signos vitales válidos (FC = 75 lpm, PAM = 85 mmHg).

Confirmación de emisión de alerta crítica en pantalla ante riesgo del 85%, con 100% de éxito.



3. Uso Asistido de Inteligencia Artificial

En cumplimiento de las pautas académicas y éticas de la asignatura Verificación y Validación de Software, el equipo Deploy & Pray declara el uso de herramientas de Inteligencia Artificial Generativa como asistencia durante la elaboración del trabajo práctico integrador.

Herramientas utilizadas:

- Antigravity.
- Claude

Alcance del uso de IA:

La herramienta se utilizó de manera acotada y puntual para las siguientes actividades de apoyo.

- **Generación de datos sintéticos de prueba**
  Prompt (Antigravity, Camila): "Generar un conjunto de valores fisiológicos sintéticos y desidentificados (frecuencia cardíaca, presión arterial media, saturación y temperatura) para simular lecturas continuas de quirófano en pacientes ficticios de 18 a 90 años, evitando el uso de información médica real según Ley 25.326."

- **Asistencia en el formateo de tablas de casos de prueba**
  Prompt (Antigravity, Camila): "Organizar en formato de tabla comparativa los casos de prueba de valores límite y partición de equivalencias previamente diseñados por el equipo, estructurando columnas de entrada, salida esperada y precondiciones."

- **Revisión gramatical y estilística**
  Prompt (Antigravity, Camila): "Revisa redacción, ortografía y estilo formal del Plan de Pruebas según terminología de la calidad (ISTQB e ISO 25010)."

  Prompt (Claude, Samuel): "informar errores gramaticales en el .feature de los escenarios propuestos"

- **Interfaz**

  Prompt (Antigravity, Camila): "Diseñar una interfaz web HTML5/CSS interactiva y responsiva que simule un monitor de signos vitales para cardiovigía, compatible con pruebas automatizadas mediante Selenium WebDriver. "

- **Pruebas de controller (Claude, Samuel)**

  Como complemento de las pruebas de API realizadas con Postman, se utilizó Claude para asistir en la creación de pruebas del controller mediante `MockMvc`*.*

  *Estos tests f*ueron creados para cubrir mediante código el mismo comportamiento de API que se valida externamente con Postman.

4. Informe Final

Auditoría clínica de datos: El equipo inspeccionó y ajustó cada lote de datos sintéticos para asegurar su concordancia estricta con los rangos definidos en los requerimientos (RF-01 a RF-11).

Control técnico: Todos los casos de prueba tabulados fueron verificados contra el comportamiento real del sistema desarrollado por el equipo en Java y Spring Boot.

Resolución propia: La totalidad de la arquitectura de pruebas, el código de las suites unitarias con JUnit 5, los escenarios BDD de Cucumber, la configuración del entorno Maven y el análisis de métricas e informe final fueron desarrollados, ejecutados y validados íntegramente por los integrantes del equipo.

4.1 Balance de Cumplimiento de Criterios de Calidad (KPIs)

A partir de la ejecución de la estrategia integral de testing sobre el backend y la interfaz de CardioVigía v2.4.0, se presenta la contrastación entre los criterios de salida (DoD) definidos en el Plan de Pruebas y los resultados reales alcanzados:

Criterio de Calidad / Métricas (DoD)	Meta Definida	Resultado Obtenido	Herramienta / Evidencia	Estado
Cobertura de Instrucciones / Líneas	≥ 80%	96%	JaCoCo 0.8.12	Superado
Cobertura de Ramas / Decisiones	≥ 75%	98%	JaCoCo 0.8.12	Superado
Calidad de Detección de Defectos (Mutation Score)	≥ 90%	98% (65/66 mutantes)	PITest 1.25.7	Superado
Eficacia de la Suite de Pruebas	100% críticos y ≥ 95% total	100% aprobadas (111 pruebas)	JUnit / Cucumber / Selenium	Cumplido
Gestión de Defectos en Jira	0 críticos abiertos y ≤ 2 altos	0 abiertos (KAN-17, 18 y 19 resueltos)	Jira	Cumplido
Trazabilidad y Nivel de Automatización	≥ 85%	100% de RF-01 a RF-11 cubiertos	BDD	Cumplido
Tiempo de Respuesta de API	< 2s P95	< 35 ms	Postman	Cumplido

4.2 Riesgos Residuales y Limitaciones del Trabajo

Aunque el software superó exhaustivamente las pruebas estáticas, dinámicas y de mutación, el equipo de QA identifica los siguientes riesgos residuales derivados de los puntos excluidos del alcance:

1. Integración con Hardware Médico Real: Las pruebas se realizaron sobre emuladores y datos sintéticos. Persiste el riesgo de ruido eléctrico no contemplado al conectar físicamente las consolas de bombas de circulación extracorpórea (CEC) reales

2. Latencia de Red Hospitalaria: Si bien los endpoints responden en < 15ms en entorno local, la latencia en quirófano dependerá de la infraestructura cableada/inalámbrica del Instituto Cardiovascular del Plata (ICP)

4.3 Lecciones Aprendidas del Proceso de QA

- El análisis de mutantes con PIT demostró ser superior a la cobertura de código tradicional: permitió descubrir que condiciones de frontera con 100% de cobertura en JaCoCo podían ocultar defectos si no se probaban los valores límite exactos

- La técnica combinatoria Pairwise con Microsoft PICT permitió reducir el esfuerzo de pruebas en un 81.5% sin sacrificar la detección de errores por interacción de parámetros

4.4 Resultados finales de ejecución de pruebas

Se ejecutó la suite completa de pruebas del proyecto mediante Maven, utilizando Java 21.

La ejecución incluyó pruebas unitarias, pruebas del controller mediante MockMvc, escenarios BDD con Cucumber y pruebas E2E con Selenium.

***Resultado*:**

- Pruebas ejecutadas: 136

- Fallos: 0

- Errores: 0

- Pruebas omitidas: 0

- Estado de la compilación: BUILD SUCCESS

Cobertura código - JaCoCo

La cobertura de código se obtuvo mediante JaCoCo luego de ejecutar la suite completa de pruebas. Los resultados superan los objetivos definidos en el plan de pruebas para líneas y decisiones.

Resultados:

- Cobertura de instrucciones: 99,3%
- Cobertura de líneas: 98,3%
- Cobertura de decisiones/ramas: 89,4%
- Cobertura de complejidad: 88,8%
- Cobertura de métodos: 98,0%
- Cobertura de clases: 100%

Objetivos del plan:

- Líneas: mínimo 80%
- Decisiones: mínimo 75%
- Caminos en código crítico: mínimo 70%

Cobertura controller - MockMvc

Se incorporaron pruebas MockMvc específicas para verificar las rutas REST del controller, incluyendo respuestas HTTP exitosas, errores, límites de valores, roles autorizados, elegibilidad, modo degradado, HCE y configuración de umbrales.

Resultados:

- Cobertura de instrucciones: 100%
- Cobertura de líneas: 100%
- Cobertura de ramas: 77,5%
- Cobertura de complejidad: 74,3%
- Cobertura de métodos: 100%

Las pruebas permiten comprobar el comportamiento HTTP del controller de forma automatizada y complementan las pruebas externas realizadas con Postman.

Mutation testing - PIT

Se ejecutó PIT para evaluar la capacidad de las pruebas automatizadas para detectar modificaciones ficticias en el código.

Resultados:

- Mutantes generados: 108
- Mutantes eliminados: 104
- Mutantes sobrevivientes: 4
- Mutation score: 96%

Objetivo del plan:

- Mutation score mínimo: 90%

Los cuatro mutantes sobrevivientes corresponden a cambios de límites condicionales en la clasificación de riesgo y en la actualización de umbrales.

Análisis de mutantes sobrevivientes

PIT detectó cuatro mutantes sobrevivientes relacionados con límites condicionales. Estos mutantes no modifican el comportamiento observado por las pruebas actuales porque no todos los valores frontera se verifican de forma individual.

Detalle:

1. CardioVigia.java, línea 36, método clasificarNivelRiesgo:

- Cambio del límite de probabilidad en la condición que valida el rango permitido.

2. CardioVigiaController.java, línea 176, método actualizarUmbrales:

- Cambio del límite inferior de umbralAlto

3. CardioVigiaController.java, línea 176, método actualizarUmbrales:

- Cambio del límite superior de umbralAlto

4. CardioVigiaController.java, línea 176, método actualizarUmbrales:

- Cambio del límite inferior de umbralCritico

Para eliminar estos mutantes se pueden agregar pruebas específicas para los valores frontera válidos: probabilidad 0.0 y 100.0, umbralAlto 40.0 y 60.0 y umbralCritico 70.0

Pruebas API - Postman

Se actualizó la colección de Postman para cubrir los requisitos funcionales RF-01 a RF-11.
La colección contiene 26 requests distribuidas entre los endpoints de signos vitales, calidad de señal, pacientes, fases quirúrgicas, elegibilidad, alertas, escalamiento, modo degradado, HCE y configuración de umbrales.

Colección:

postman/CardioVigia_Postman_Collection.json

Entorno:

postman/CardioVigia_Environment.json

Nota:

Las pruebas Postman validan el comportamiento externo de la API, pero no la generan cobertura JaCoCo directamente.

Pruebas end-to-end - Selenium

Se ejecutaron cuatro pruebas E2E sobre la interfaz web utilizando Selenium WebDriver y Chrome.
Las pruebas verifican la validación de signos vitales, el rechazo de valores fuera de rango, la generación de alertas críticas y el rechazo de probabilidades inválidas.

Resultados:

- Pruebas ejecutadas: 4
- Fallos: 0
- Errores: 0

Pruebas BDD - Cucumber

Se ejecutaron los escenarios BDD definidos en el feature de CardioVigia. La suite verifica los requisitos funcionales RF-01 a RF-11 mediante escenarios escritos en lenguaje Gherkin y step definitions Java.

Resultados:

- Escenarios ejecutados: 97
- Fallos: 0
- Errores: 0

Feature principal:

Pruebas unitarias

Las pruebas unitarias verifican la lógica principal de CardioVigia, incluyendo validación de signos vitales, clasificación de riesgo, notificaciones, escalamiento, elegibilidad y modo degradado.

Resultados:

- Pruebas ejecutadas: 14
- Fallos: 0
- Errores: 0

5. Dictamen del Equipo de QA

Recomendación final: LIBERAR CON CONDICIONES

Fundamentación Técnica: El sistema CardioVigía v2.4.0 cumple de manera rigurosa con los atributos de calidad de Funcionalidad, Confiabilidad y Seguridad establecidos bajo el estándar ISO/IEC 25010. La lógica de discriminación de ruido, supresión de fatiga de alarmas (120 s), escalamiento por temporizadores y conmutación segura a modo degradado ante fallas se encuentra 100% verificada mediante pruebas unitarias, BDD y de API.

Condiciones Obligatorias para el Pase a Producción:

1. Desplegar la versión 2.4.0 en el ambiente de Staging hospitalario
2. Ejecutar la batería de pruebas de carga conectando en simultáneo las 24 consolas físicas de quirófano para validar la concurrencia en tiempo real antes de su uso con pacientes